

Sistemas Operativos

2º Parcial 2C2024 – TT – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1. Son TLB y MMU. Ambos agilizan y facilitan la traducción de direcciones.
TLB guarda porciones de las tablas de páginas que en caso de un acierto de TLB, permite reducir el tiempo en la traducción de una dirección lógica. Las consultas en la TLB tienen un tiempo de respuesta más bajo que realizarlas en la tabla de páginas. Si no hubiera TLB, todas las consultas deberían realizarse en la tabla de páginas.
MMU es la encargada de realizar las traducciones. Al ser un componente del hardware, la traducción es más rápida y libera al sistema operativo de realizar esa función. En caso que no estuviese presente, esta tarea debería hacerla el sistema operativo, agregando mucho overhead.

2.

	Overhead	Fragmentacion
Particionamiento fijo	Poco	Interna
Particionamiento dinámico	Alto (considerar compactación)	Externa
Buddy System	Medio/Alto	Interna y externa

3.

Trashing: fenómeno en el cual durante un intervalo de tiempo, la cpu pasa mas tiempo con el SO realizando tareas administrativas relativas a fallos de página que ejecutando instrucciones de procesos (procesamiento útil)

El nivel de multiprogramación podría incrementarse en un contexto donde el sistema se encuentre siendo subutilizado, en particular con bajo consumo de CPU. Inicialmente se podrían obtener mejoras, pero si el nivel de multiprogramación se continuara incrementando, eventualmente podría desencadenar en thrashing, dado que por cada proceso de dispondrían muy pocas páginas en memoria.

4. La llamada a write() respondería con un error o se bloquearía, y la escritura no se realizaría. Se encontrarían involucradas la tabla global de archivos abiertos, donde se ubican los permisos para acceder al archivo y la información relativa a la existencia del lock sobre dicho archivo.
5. Disponer de punteros directos e indirectos de distintos niveles permite poder trabajar eficientemente tanto con archivos chicos (ocupando los bloques indispensables y brindando acceso rápido) como trabajar con archivos grandes (pudiendo expandir el inodo a demanda). Disponer solamente de punteros indirectos triples generaría accesos mas lentos a archivos chicos, y mas bloques ocupados. Finalmente, podrían liberarse mas bloques que los de datos si alguno de los bloques de datos se encontrara en algún nivel de indirección..

Práctica

1)

a)

Para el requerimiento de que la tabla no crezca de tamaño sin importar la cantidad de procesos o su tamaño, la tabla de páginas invertida puede ser una buena opción (descontando sus problemas asociados a su uso en memoria virtual).

Si la ram es de 20MiB y los frames de 1Kb, entonces $20\text{MiB} / 1\text{KiB} = 20480$

La tabla debería tener:

- Para el pid: 8 bits ($2^8 = 256$)
- Para el # pagina: 10 bits (de los 20 bits de las direcciones, los 1eros 10 son para el numero de pagina)
- Para el puntero al siguiente frame: 15 bits ($2^{15} = 20480$)

# frame	pid	# pagina	Puntero stg frame
-	8 bits	10 bits	15 bits

b)

Un fallo de página se determinaria luego de recorrer la tabla y habiendo agotado los lugares posibles (el frame designado para ubicar el frame, y los frames designados como "siguientes"), no se hubiera encontrado la página para el pid deseado.

Un ejemplo con 4 accesos podría ser:

1. Acceso a tabla: no se encuentra la página en frame deseado, no hay siguiente. Fallo de pagina.
2. Acceso a tabla: Luego de cargarse la pagina, se accede a la tabla nuevamente, el frame está ocupado por otro proceso.
3. Acceso a tabla: Se avanza al siguiente y se logra encontrar la pagina
4. Acceso al frame propiamente dicho

2) Datos:

Disco: 32 TiB

Tamaño de bloque: $4 \text{ KiB} = 2^2 * 2^{10} \text{ Bytes} = 2^{12} \text{ Bytes}$

Tamaño puntero = 32 bits = 4 Bytes = 2^2 Bytes

Cantidad máxima de punteros = 2^{32}

Header: $48 \text{ KiB} / 4 \text{ KiB} = 12 \text{ bloques}$

Main: $16 \text{ MiB} / 4 \text{ KiB} = 4096 \text{ bloques}$

a) Cantidad de punteros x bloque = tam bloque / tam puntero

$2^2 * 2^{10} / 2^2 = 2^{10} \rightarrow$ **1024 punteros por bloque**

12 punteros directos $\rightarrow 48 \text{ Kib}$

1 puntero indirecto simple $\rightarrow 1024 * 4\text{Kib} \rightarrow 4\text{Mib}$

1 puntero indirecto doble $\rightarrow 1024 * 1024 * 4\text{Kib} \rightarrow 4\text{Gib}$

Partimos desde el bloque 8,5 $\Rightarrow$ **$34816 / (4*1024) = 8.5 \Rightarrow$ Nro 8**

Asumiendo el bloque nro 8 está cargado en memoria, faltan cargar 3 bloques para completar los 48 KiB.

El main se carga usando los PI:

$16 \text{ MiB} = 16 * 2^{20} \text{ Bytes} = 2^{24} \text{ (4096 bloques)}$

Va a ocupar 4096 bloques de datos:

1024 direccionados por el IS $\rightarrow$ **1 acceso al Bloque de Punteros y 1024 a BD**

3072 usando IDs $\rightarrow$ **1 acceso a BP, 3 accesos a BP y $3*1024$ a BD**

Sumatoria total: $3 \text{ (datos)} + 1 + 1024 \text{ (datos)} + 1 + 3 + 3 * 1024 \text{ (datos)} =$ **4104 bloques accedidos de disco.**

b)

12 punteros directos $\rightarrow 48\text{Kib}$

1 puntero indirecto simple $\rightarrow 4\text{Mib}$

1 puntero indirecto doble $\rightarrow 4\text{Gib}$

Archivo **48KiB, 8MiB y 4GiB**

Con la conformación original no alcanzaría para direccionar todo el nuevo archivo (faltan 4 MiB). Para direccionar los bytes que faltan, el cambio que implicaría agregar la menor cantidad de accesos a disco sería agregar un puntero indirecto simple.

c)

Teórico: $2^{32} * 2^2 * 2^{10} = 2^{45} = 16 \text{ TiB}$

Físico: 32 TiB

Real: 16 TiB

Podría duplicar el tamaño de bloque para utilizar la capacidad completa del disco aunque aumentamos la fragmentación externa en consecuencia.

3)

a) Los bloques son de 4 bytes cada uno.

Al ocupar 20 bytes el archivo necesitará 5 clusters: $20 \text{ bytes} / 4 \text{ bytes} = 5$

Las secuencias posibles leyendo la tabla FAT son:

- a. $15 \rightarrow 2 \rightarrow 9$ (ESTE_NO_ES)
- b. $12 \rightarrow 14 \rightarrow 4$ (ACÁ_NO_ESTÁ!)
- c. $5 \rightarrow 11 \rightarrow 10$ (KERNEL_PANIC)
- d. $3 \rightarrow 8 \rightarrow 13 \rightarrow 6 \rightarrow 1$ (SISTEMAS_OPERATIVOS!)

0	0	@@@@
1	FIN	VOS!
2	9	_NO_
3	8	SIST
4	FIN	stá!
5	11	KERN
6	1	RATI
7	error	AFER
8	13	EMAS
9	FIN	ES
10	FIN	ANIC
11	10	EL_P
12	14	acá_
13	6	_OPE
14	4	no_e
15	2	ESTE

b) En FAT 32, el tamaño máximo de un archivo coincide con el tamaño máximo del file system. Usamos 28 bits por puntero, por lo que habrá 2^{28} entradas:

Tamaño máximo de un archivo = cantidad de punteros (clusters direccionables) * tamaño de clusters = $2^{28} \text{ entradas} \times 4 \text{ bytes} = 2^{28} \times 2^2 = 1 \text{ GiB}$.

$1 \text{ GiB} < 1 \text{ TiB}$

El tamaño máximo real y teórico de un archivo y del file system es 1GB

c) Tamaño teórico FAT = cantidad de punteros * tamaño puntero = $2^{28} \times 4 \text{ bytes} = 2^{28} \times 2^2 = 1 \text{ GiB}$.

Cada tabla FAT ocupa 1 GiB.

d) El espacio utilizado por la tabla FAT coincide con el espacio disponible para almacenar información. Peor sería el caso, si existiesen más copias de la tabla FAT. Dado que el disco es de 1 TiB y no puede aprovecharse por completo, sería conveniente que los cluster sean más grandes. Con cluster de 4 KiB podría aprovecharse todo el disco.